



عملي

4

12

1440 sp

كلية الهندسة المعلوماتية

السنة الثالثة

تصميم وحدة التحكم (2) Control Unit



RB Informatics; 05/11/2024

عملي مشترك

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحواسيب 2

سنكمل في هذه المحاضرة حل مسألة تصميم المعالج.

ارسم ممر المعطيات Data path وحدد إشارات التحكم المطلوبة، ثم استنتج معادلاتها.

نريد رسم ممر المعطيات حسب التعليمات التي أخذناها سابقاً:

1	F1	AR ← PC	9	SUBM-D3	DR ← AC
2	F2	DR ← M[AR] ; PC ← PC + 1	10	SUBM-D4	M[AR] ← DR
3	F3	IR ← DR(op) ; AR ← DR(x)	11	LDR-ID1	DR ← M[AR]
4	OR-R	AC ← AC or R	12	LDR-ID2	AR ← DR
5	LDR-D1	DR ← M[AR]	13	LDR-ID3	DR ← M[AR]
6	LDR-D2	R ← DR	14	LDR-ID4	R ← DR
7	SUBM-D1	DR ← M[AR]	15	JMP	PC ← DR(x)
8	SUBM-D2	AC ← DR - AC	16	NOP	Do nothing



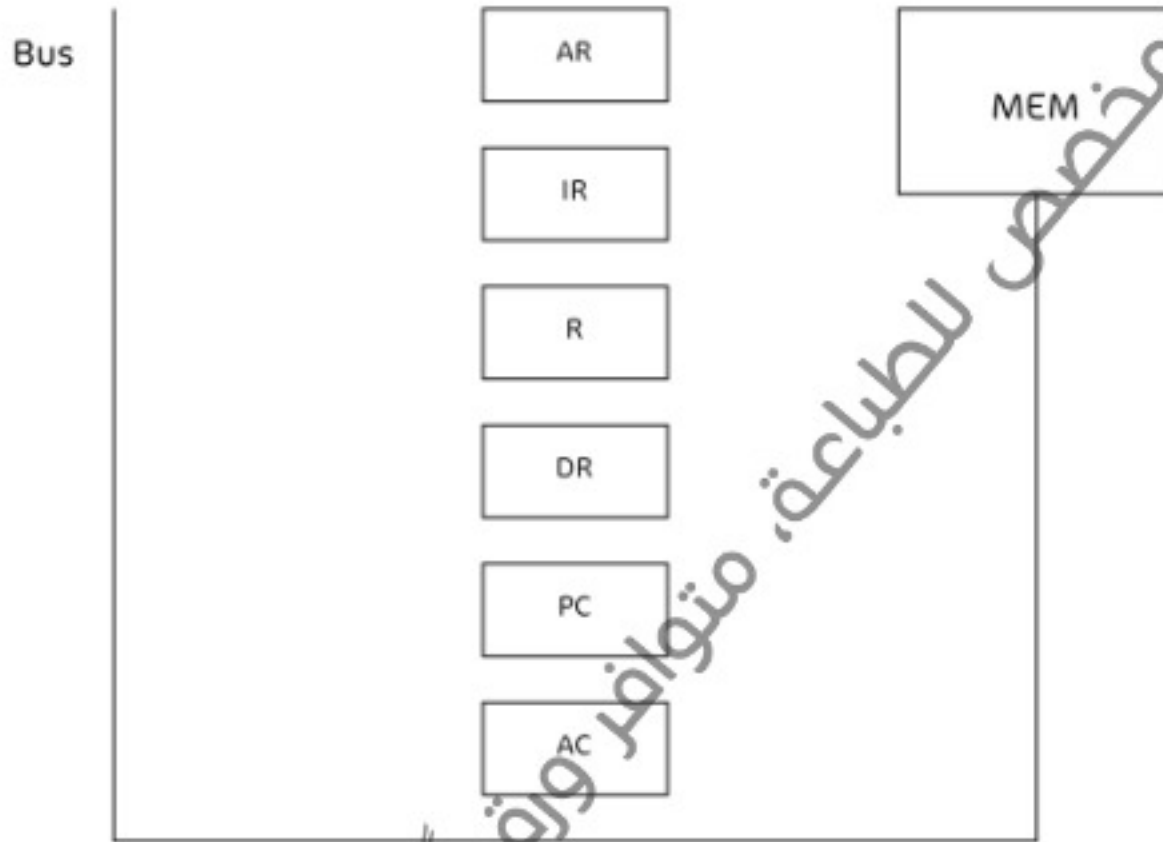
Let's get started

والرسم تتبع الخطوات التالية:

- بداية نرسم المعطيات الأساسية:
- رسم المسار Bus الذي ستنتقل عبره محتويات السجلات.
- رسم السجلات الـ 6 المستخدمة:

AR, IR, R, DR, PC, AC

- رسم الذاكرة:



- وصل الخطوط حسب التعليمات:

اعتماد النقل من اليسار إلى اليمين

1. $F1: AR \leftarrow PC$

لدينا وصلة من PC إلى AR لكن لا نستطيع الوصل مباشرة فنوصل PC إلى Bus (الذي نعتبره كالاتسترد) و Bus إلى AR وذلك لتنظيم عملية النقل.

2. $F2: DR \leftarrow M[AR] ; PC \leftarrow PC + 1$

بداية نصل AR إلى الذاكرة Mem مباشرة وبعدها نصل وصلة إلى DR. $PC \leftarrow PC + 1$ في هذه العملية أقوم بوضع دائرة لزيادة العداد PC.

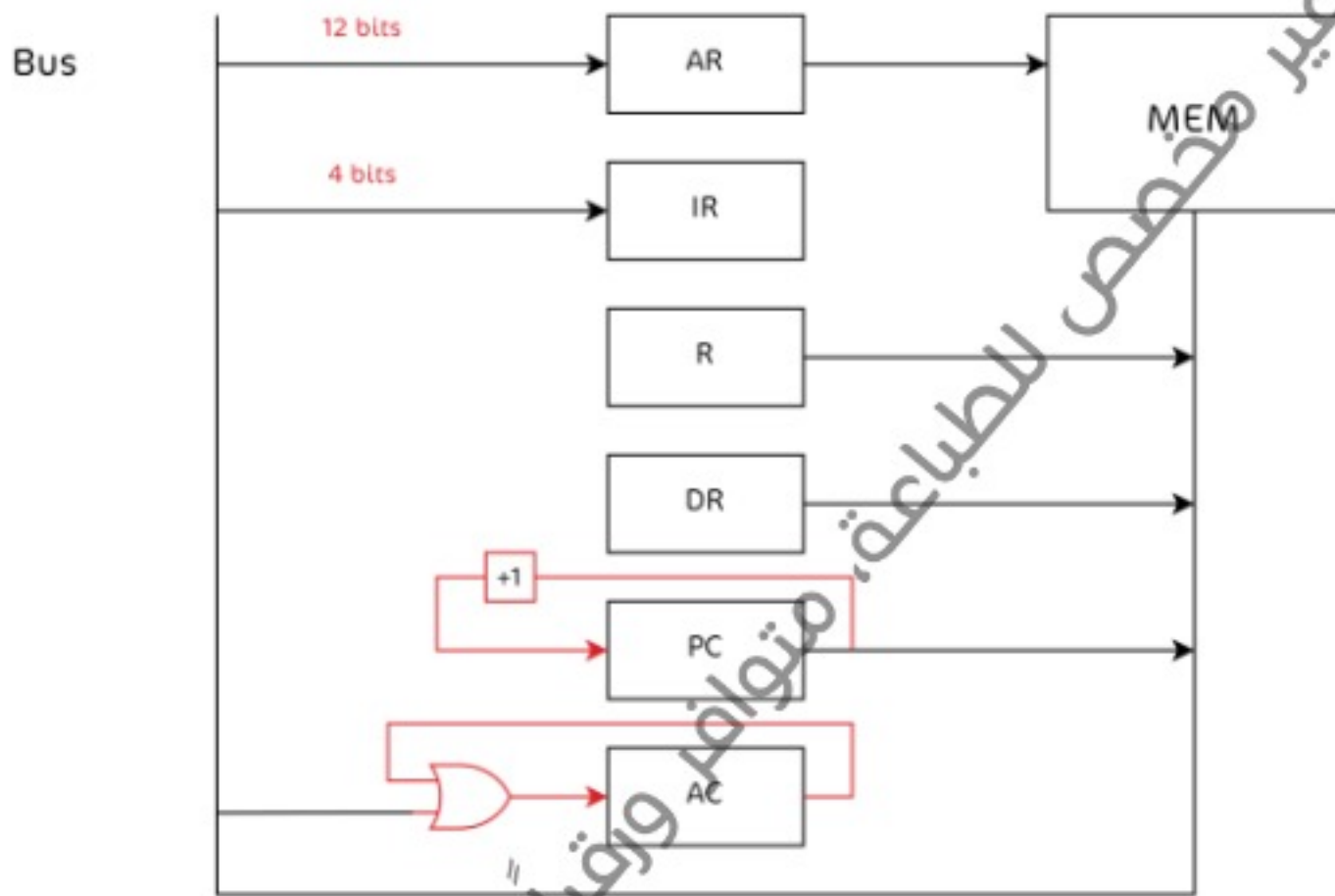
3. $F3: IR \leftarrow DR(op) ; AR \leftarrow DR(x)$

نخرج وصلة من DR إلى Bus ومن Bus إلى IR وهنا نحتاج إلى 4 bits، وكذلك نخرج وصلة من DR إلى Bus (وهي موجودة مسبقاً) ومن Bus إلى AR (وهي موجودة مسبقاً) وهنا نحتاج إلى 12 bits.

4. OR-R: $AC \leftarrow AC \text{ or } R$

نوصل وصلة من R إلى Bus ومن Bus إلى AC وكذلك نوصل وصلة من خرج AC إلى بوابة or وذلك لنقوم بعملية or بين AC و R.

■ فيصبح ممر المعطيات:



5. LDR-D1: $DR \leftarrow M[AR]$

(موجودة مسبقاً).

6. LDR-D2: $R \leftarrow DR$

نوصل وصلة من DR إلى Bus (موجودة مسبقاً) ومن Bus إلى R.

7. SUBM-D1: $DR \leftarrow M[AR]$

(موجودة مسبقاً).

8. SUBM-D2: $AC \leftarrow DR - AC$

وصلة DR إلى Bus موصولة ولكن في هذه المرحلة أصبح لدينا وصلتين على مدخل AC لكن لا نستطيع وصلهما كلاهما لذلك نضع Multiplexer الناخب، وهو أنه لدينا أكثر من دخل ونحتاج إلى خرج واحد وهنا لدينا دخلين فنمثلهما بـ 1 bit، والذي سيحدد الخرج هو خط التحكم فمثلاً نعتبر 1 يمثل خرج الطراح و 0 يمثل خرج بوابة or، ومن الذي يحدد إذا كنا نريد 0 أو 1؟

Control Unit التي نصممها

9. SUBM-D3: $DR \leftarrow AC$

وصلة من AC إلى Bus ووصلة من Bus إلى DR (موجودة مسبقاً)

10. SUBM-D4: $M[AR] \leftarrow DR$

وصلة DR (موجودة مسبقاً) ووصلة Bus إلى Memory.

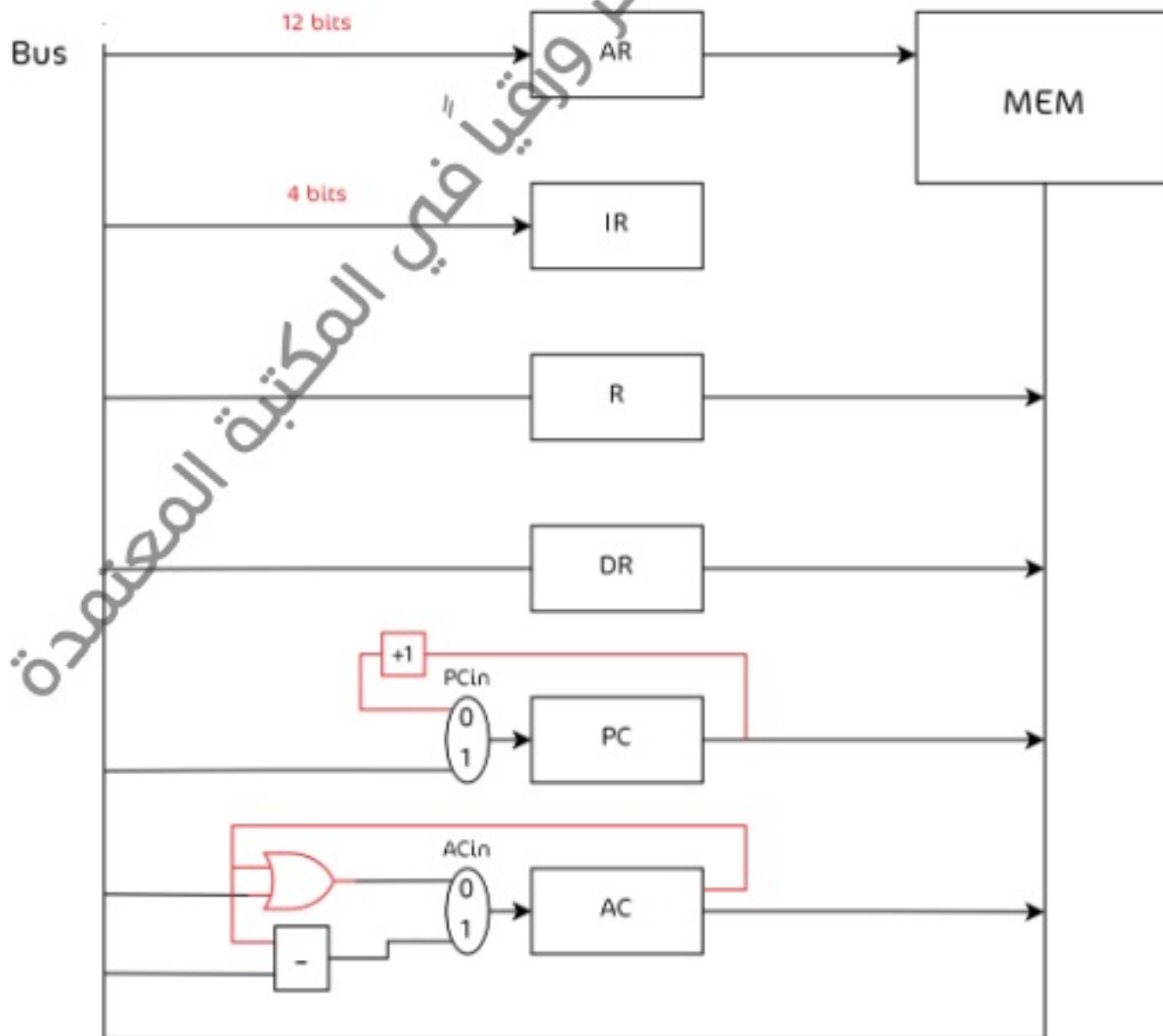
التعليمات 11 + 12 + 13 + 14 موصولين مسبقاً.

15. JMP: $PC \leftarrow DR(x)$

نفس الحالة عند السجل AC حيث أصبح لمدخل PC وصلتان ووصلة من $DR(x)$ ووصلة دائرة زيادة العداد PC لذلك نضع أيضاً Multiplexer في حالة 1 الخرج هو $DR(x)$ ، في حالة 0 الخرج هو دائرة زيادة العداد PC.

16. NOP: Do nothing

لا تقوم بفعل شيء.



هكذا نكون انتهينا من وصل الوصلات لكن لدينا بعض المشاكل علينا حلها:

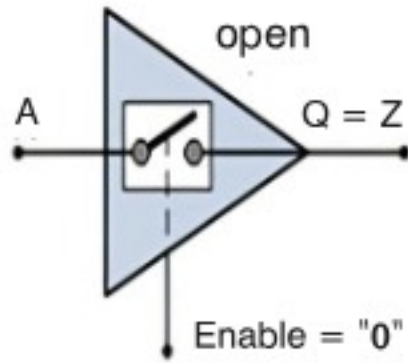
- أكثر من خط يصبون على ال Bus ولتنظيم هذه العملية ومعرفة ما يحتويه ال Bus نستخدم ما يسمى ب Tri-state Buffer.

ال Buffer يعمل بعكس العاكس أي:

- الدخل 0 ← الخرج 0

- الدخل 1 ← الخرج 1

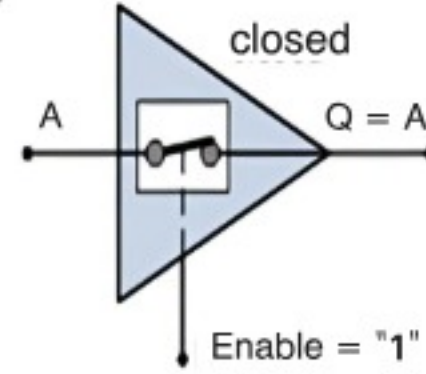
ويحتوي ال Buffer خط تحكم إذا كان 0 أي ال Buffer غير مفعّل فيصبح مثل قاطع مفتوح أي لا يمرر قيمة أما إذا كان 1 أي ال Buffer مفعّل ويمرر قيمة:



High impedance

خط التحكم 0،

ال Buffer غير مفعّل ولا يمرر قيمة



Buffer

خط التحكم 1،

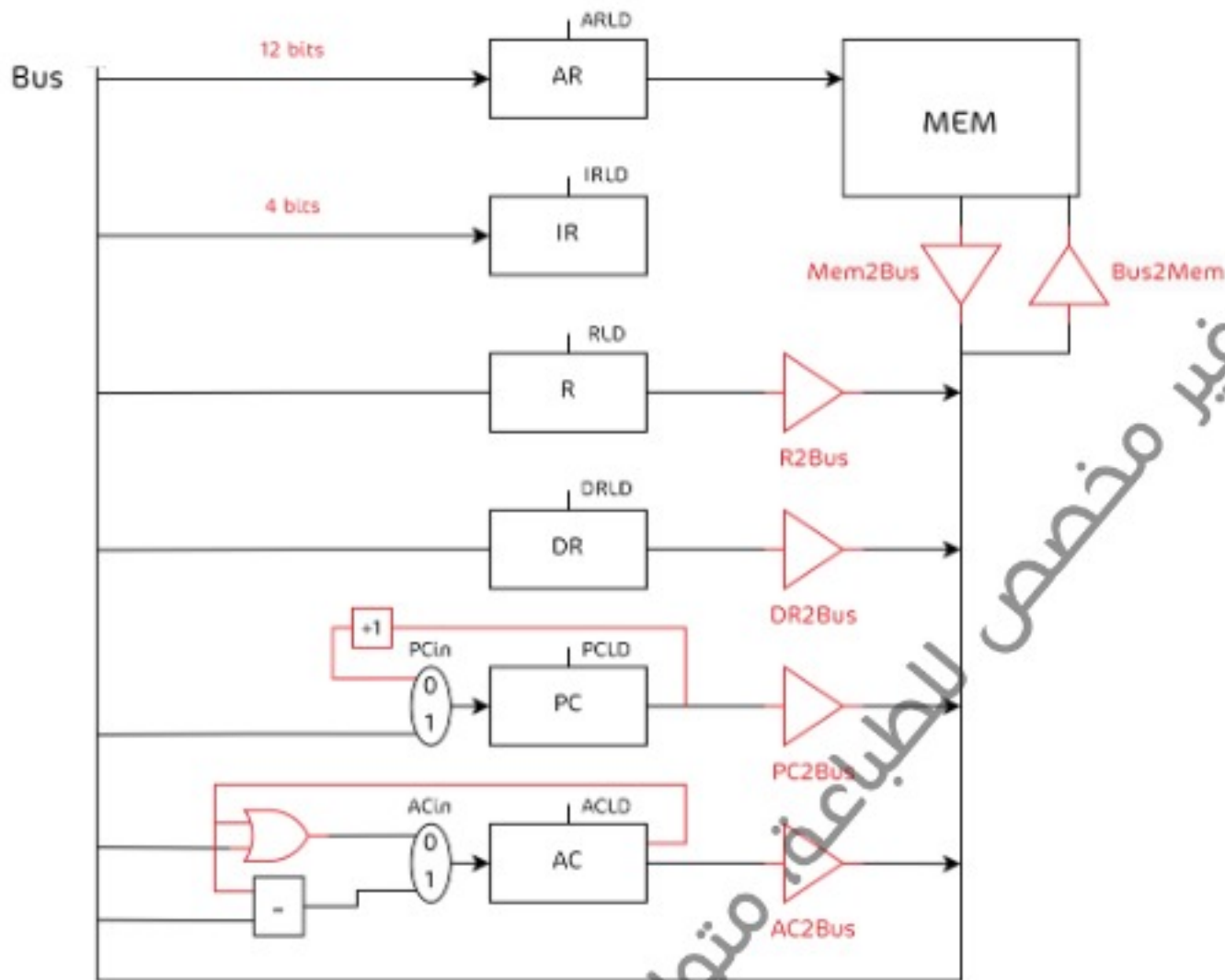
ال Buffer مفعّل ويمرر قيمة

- بهذه الحالة عندما نريد قيمة سجل معين نحول ال Buffer الخاص به إلى 1 والباقي 0، بالتالي نضع Tri-state Buffer على كل خط واصل إلى Bus، باستثناء خط AR لأنه موصول إلى Mem وليس إلى Bus وكذلك IR لأنه بالأصل ليس له وصلة إلى Bus.

عندما تخرج قيمة معينة إلى Bus كيف سيحدد إلى أي سجل ستدخل؟

- لدينا في السجلات إشارة تحكمية (خط تحكمي) يسمى LD/WR عند الكتابة على السجل تكون قيمته 1 وعند عدم الكتابة تكون قيمته 0.

وبرسم ممر المعطيات بعد إضافة خطوط و إشارات التحكم تصبح:



معادلات إشارات التحكم

لاستخراج معادلات LD تكون عبارة عن المراحل التي تكون فيها الكتابة على السجل (أي المرحلة التي يكون فيها السهم داخل إلى السجل).

$$PCLD = F2 + JMP$$

$$RLD = LDR-D2 + LDR-ID4$$

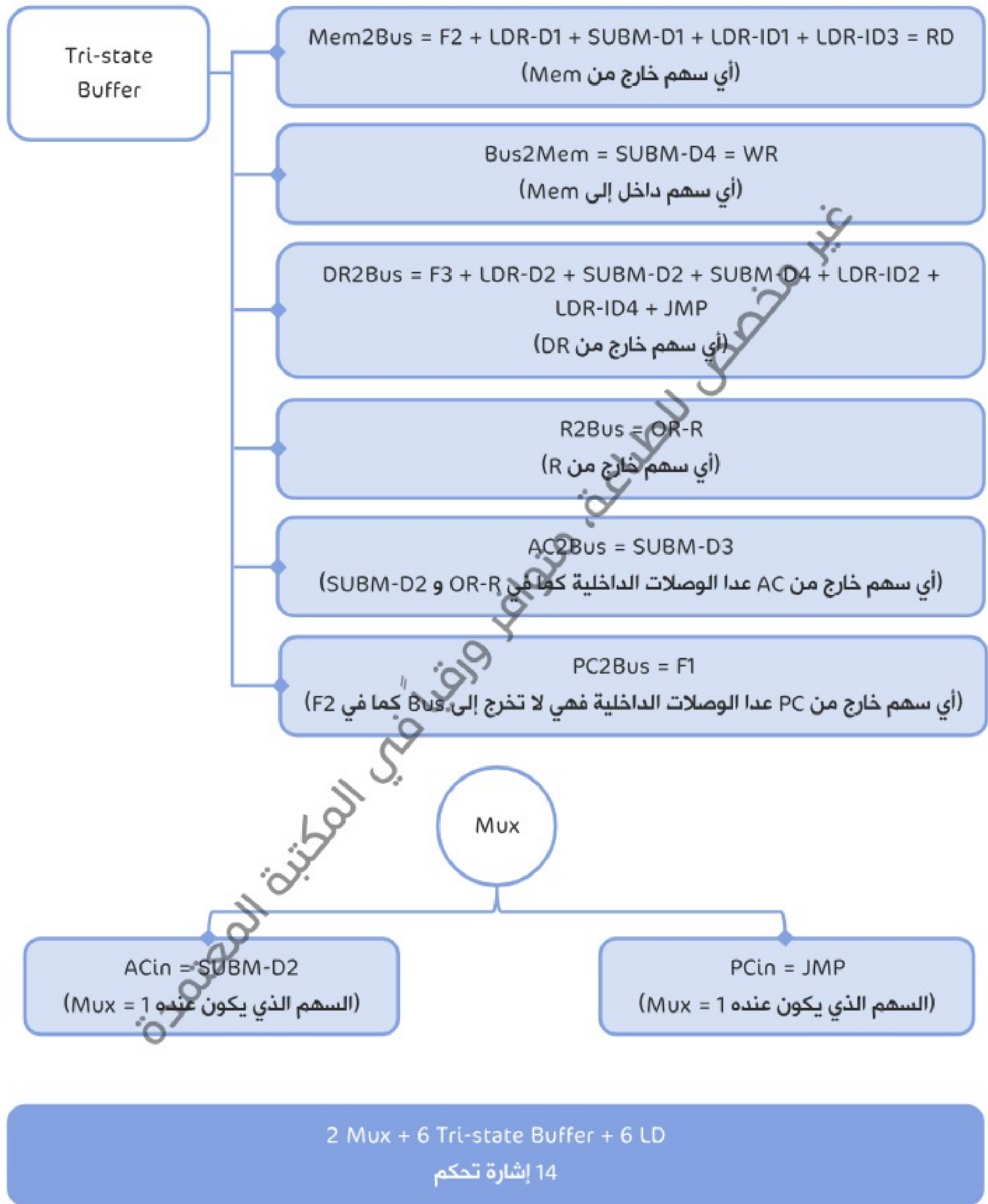
$$ACLD = OR-R + SUBM-D2$$

LD

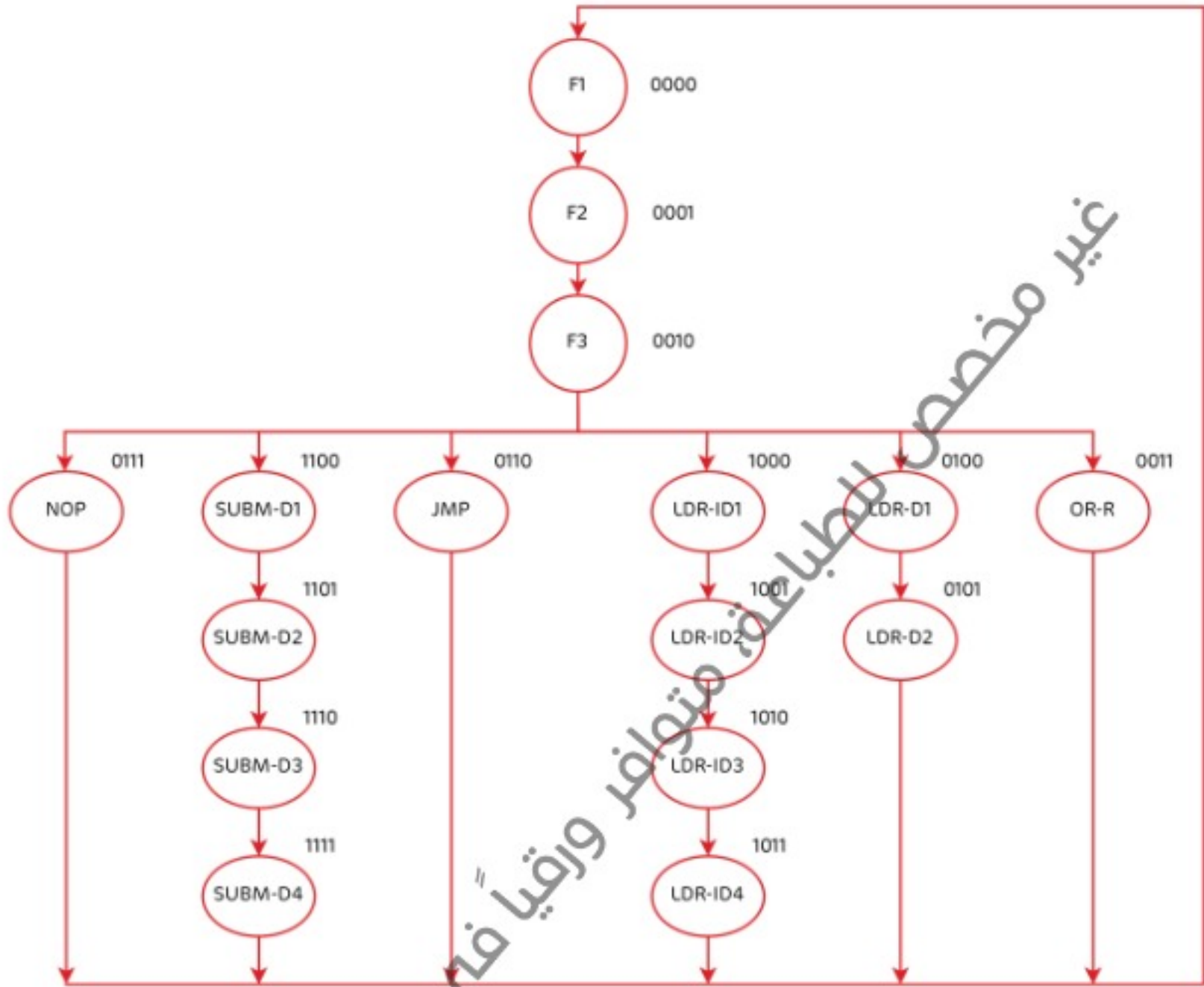
$$ARLD = F1 + F3 + LDR-ID2$$

$$DRLD = F2 + LDR-D1 + SUBM-D1 + SUBM-D3 + LDR-ID1 + LDR-ID3$$

$$IRLD = F3$$



ارسم مخطط الحالات مع ترميز الحالات:



نلاحظ أن Fetch مشتركة لجميع التعليمات.

■ اختيار الترميز للحالات:

نستخدم counter لترميز الحالات، وبما أنه لذي 16 حالة فهي ترمز على 4 bits.
 F1: الترميز صفر 0000 دائماً لأنه عند انتهاء أي تعليمة نضع العداد فنعود لحالة F1
 F2: زيادة 1 عن F1 لأنها حالة تليها فيصبح الترميز 0001.
 F3: زيادة 1 عن F2 لأنها حالة تليها فيصبح الترميز 0010.

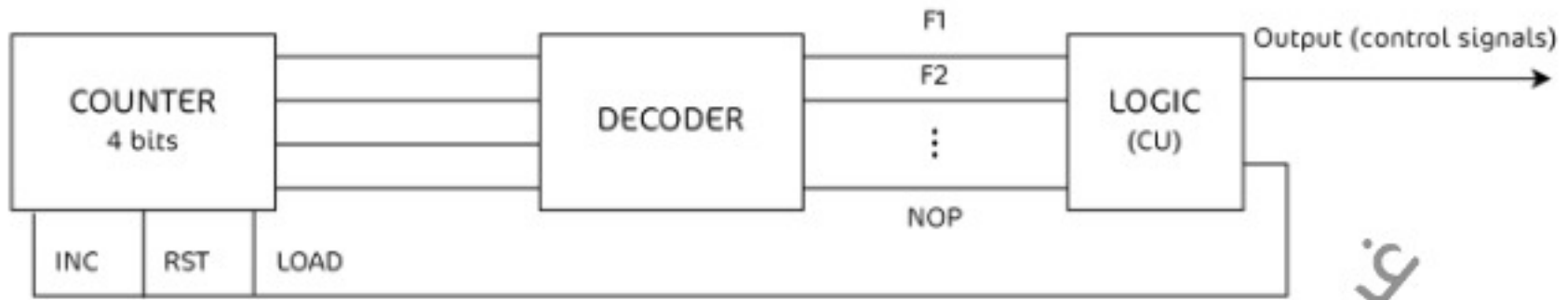
■ الترميز في الفروع اختياري مع مراعاة:

عدم ترميز حالتين بنفس الرمز
 مراعاة تسلسل المراحل ضمن التعليمة، مثال:

LDR-ID1: 1000
 LDR-ID2: 1001 الحالة التالية.

في نهاية أي تعليمة يوجد سهم يعيدنا إلى F1 هذه يدل على تصفير العداد.

ارسم التصميم العتادي لوحدة التحكم Hardwired CU، واستنتج معادلات إشارات التحكم INC, LOAD, RST:



- Counter: بنيته الداخلية عبارة عن قلابات لذلك استعصنا عن القلابات به.
- Decoder: دخله 4 bits حيث 4 لترميز الحالات، وخرجه 16 واحد فقط مؤهل.
- LOGIC: خرج البوابات المنطقية إشارات تحكم:



INC

تسلسل الحالات

RST

عند انتهاء التعليمة (آخر مرحلة من التعليمة)

LOAD

عندما تنتهي F3 ويكون تفرع

مثلاً:



معادلات إشارات التحكم بالعداد:

- نحصل على إشارة تصفير العداد RESET بأخذ الحالات النهائية لكل تعليمة:

$$RST = NOP + SUBM-D4 + JMP + LDR-ID4 + LDR-D2 + OR-R$$

- نحصل على إشارة التحميل LD من مكان وجود التفرع في مخطط الحالات المنتهية، وفي حالتنا فإن:

$$LOAD = F3$$

- إشارة زيادة العداد INC نحصل عليها من الحالات التي يزيد فيها العداد بمقدار 1 للانتقال إلى الحالة المستقبلية، وهي جميع الحالات ماعدا الحالات النهائية ومكان وجود التفرع، فهي إذاً:

$$INC = F1 + F2 + SUBM-D1 + SUBM-D2 + SUBM-D3 + LDR-ID1 + LDR-ID2 + LDR-ID3 + LDR-D1$$

تصميم وحدة التحكم بطريقة البرمجة الصغيرة Micro-programmed CU

في الطريقة البرمجية نستعير عن الدارات المنطقية بذاكرة صغيرة Micro Memory للحصول على إشارات التحكم.

- تعليمية صغيرة Micro Instruction $\Leftrightarrow$ المرحلة (مثل F1).
- عملية صغيرة Micro Operation $\Leftrightarrow$ عملية النقل (مثل $DR \leftarrow M$).
- (التعليمية الصغيرة مؤلفة من عملية صغيرة أو أكثر)
- البرنامج الصغير Micro Program $\Leftrightarrow$ يتألف من تسلسل التعليمات الصغيرة (كلمات التحكم)، يخزن في الذاكرة الصغيرة.

صيغة التعليمات الصغيرة

أنواع الترميز

الترميز الشاقولي Vertical Coding

الترميز الأفقي Horizontal Coding

الترميز المباشر Direct Coding

ترميز العمليات الصغيرة ثنائياً

لكل عملية صغيرة بت منفصل

لكل إشارة تحكم بت منفصل

يرد في بعض المراجع أن الترميز نوعان:

- الأفقي وهو ما يكون فيه لكل إشارة تحكم بت منفصل.
 - الشاقولي وهو ما يتم فيه ترميز العمليات الصغيرة.
- أما هنا سنعتمد التقسيمات الثلاثة المذكورة بالأعلى.

التصميم البرمجي (البرمجة الصغيرة)

صيغة التعليمية الصغيرة: يتألف كل سطر في الذاكرة الصغيرة من ثلاثة حقول:

مصدر العنوان

Select Address

العنوان

Address

حقل التحكم / العملية الصغيرة

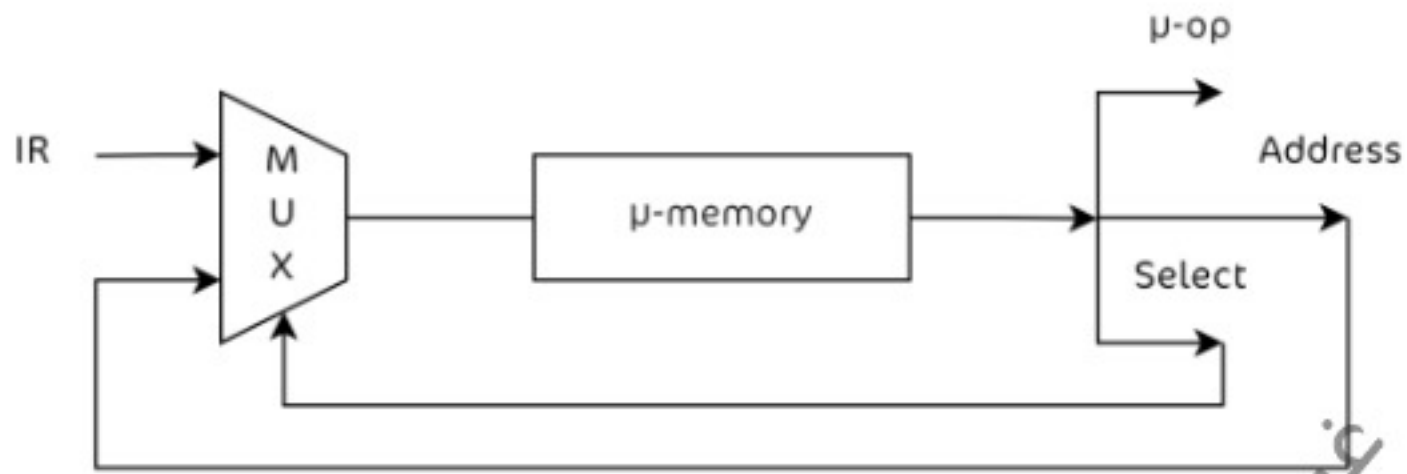
CONTROL / μ op

حقل الاختيار Select: يتألف هنا من 1 بت:

مثلاً قيمته 0 عندما يكون عنوان التعليمية التالية في الذاكرة الصغيرة، 1 مجال وجود تفرع.

حقل العنوان Address: يمثل عنوان التعليمية الصغيرة التالية للتنفيذ.

حقل التحكم (أو العملية الصغيرة CONTROL / μ op): يختلف محتوى هذه الحقل حسب الطريقة المتبعة في الترميز.



تمرین

ماهو حجم ذاكرة التحكم الصغرية اللازمة من أجل المسألة السابقة بفرض استخدام الترميز المباشر؟

عدد أسطر الذاكرة الصغرية = عدد التعليمات الصغرية = (عدد كلمات التحكم) = (عدد الحالات) = 16 سطر.

■ لدينا في كل سطر:

Select = 1 bit

Address = 4 bits (for 16 micro inst)

CONTROL FIELD = 14 bits (for 14 control signals)

(يخصص في حقل التحكم بت واحد لكل إشارة تحكم، تكون قيمتها 1 عندما يريد إصدار هذه الإشارة، و 0 عندما لا يريد إصدارها)

حجم الذاكرة الصغرية:

$$304 \text{ بت} = (1 + 4 + 14) * 16$$

OR-R: $AC \leftarrow AC \text{ or } R$

	SELECT	ADDRESS	CONTROL FIELD
			ARLD DRLD IRLD PCLD RLD ACLD M2Bus Bus2M DR2Bus R2Bus AC2Bus PC2Bus PCin ACin
F1	0	0001	100000 000001 00
F2	0	0010	010100 100000 00
F3	1	xxxx	101000 001000 00
OR-R	0	0000	000001 000100 00

ما هو حجم ذاكرة التحكم الصغرية اللازمة للمسألة السابقة بفرض استخدام الترميز الأفقي؟

الاختلاف عن الطلب السابق بحجم حقل التحكم CONTROL FIELD حيث فيه بت لكل عملية صغرية. نضع لائحة بتسميات للعمليات الصغرية (بدون تكرار)، مثلاً:

{ARPC, DRM, INCPC, IRDR, ARDR, OR-R, RDR, SUB, DRAC, MDR, PCDR, NOP}

نلاحظ أن لدينا 12 عملية صغرية مختلفة، ويخصص لكل منها بت، وبالتالي فإن عرض السطر كما يلي:

Select 1 bit	Address 4 bit	CONTROL FIELD 12 bit
-----------------	------------------	-------------------------

حجم الذاكرة الصغرية:

$$272 = (1 + 4 + 12) * 16 \text{ بت}$$

هنا يلزم دائرة منطقية إضافية لاستنتاج إشارات التحكم.

ما هو حجم ذاكرة التحكم الصغرية اللازمة من أجل المسألة السابقة بفرض استخدام الترميز الشاقولي؟

يختلف عن التمرين السابق بحجم حقل التحكم CONTROL FIELD حيث يتم ترميز العمليات الصغرية ثنائياً. نضع لائحة بتسميات للعمليات الصغرية (بدون تكرار)، مثلاً:

{ARPC, DRM, INCPC, IRDR, ARDR, OR-R, RDR, SUB, DRAC, MDR, PCDR, NOP}

وهي 12 عملية صغرية وبالتالي نحتاج إلى 4 bits للترميز، لكن بما أنه لدينا بعض التعليمات الصغرية مؤلفة من عمليتين صغريتين معاً (أي نريد تنفيذ كلتا العمليتين معاً)، لذا يتم تقسيم العمليات الصغرية إلى مجموعتين مستقلتين، بشرط عدم تواجد عمليتين صغريتين على التوازي في المجموعة نفسها، ونضع حالة NULL في كلتا المجموعتين لأنه في حالة كان لدينا عملية واحدة تنفذ في مجموعة ولا يقابلها عملية في مجموعة أخرى (أي لا يوجد عملية معها على التوازي) فنضع NULL في هذه الحالة، وبالتالي يكون لدينا المجموعتين:

- الأولى G1: تضم مثلاً INCPC, ARDR, NULL يلزمها للترميز 2 bits.
- الثانية G2: تضم باقي العمليات بالإضافة إلى حالة NULL يلزمها للترميز 4 bits.

حجم الذاكرة الصغرية:

$$176 = (1 + 4 + 6) * 16 \text{ بت}$$

هنا يلزم دائرة منطقية إضافية ومفككات ترميز لاستنتاج إشارات التحكم.



The End